SN Computer Science (2025) 6:282
https://doi.org/10.1007/s42979-025-03790-z

ORIGINAL RESEARCH

Efficient Global Neural Architecture Search

Shahid Siddiqui[1] · **Christos Kyrkou**[2] · **Theocharis Theocharides**[1,2]

Received: 4 March 2024 / Accepted: 8 February 2025
© The Author(s) 2025

Abstract
Neural architecture search (NAS) has shown promise towards automating neural network design for a given task, but it is computationally demanding due to training costs associated with evaluating a large number of architectures to find the optimal one. To speed up NAS, recent works limit the search to network building blocks (modular search) instead of searching the entire architecture (global search), approximate candidates' performance evaluation in lieu of complete training, and use gradient descent rather than naturally suitable discrete optimization approaches. However, modular search does not determine network's macro architecture i.e. depth and width, demanding manual trial and error post-search, hence lacking automation. In this work, we revisit NAS and design a navigable, yet architecturally diverse, macro-micro search space. In addition, to determine relative rankings of candidates, existing methods employ consistent approximations across entire search spaces, whereas different networks may not be fairly comparable under one training protocol. Hence, we propose an architecture-aware approximation with variable training schemes for different networks. Moreover, we develop an efficient search strategy by disjoining macro-micro network design that yields competitive architectures in terms of both accuracy and size. Our proposed framework achieves a new state-of-the-art on EMNIST and KMNIST, while being highly competitive on the CIFAR-10, CIFAR-100, and FashionMNIST datasets and being 2-4 times faster than the fastest global search methods. Lastly, we demonstrate the transferability of our framework to real-world computer vision problems by discovering competitive architectures for face recognition applications.

Keywords Neural architecture search · AutoML · Deep learning · Image classification · Convolutional neural networks

Introduction

Neural Architecture Search (NAS) is the task of automating the neural network design process for a given dataset. NAS works based on reinforcement learning (RL) and evolution (EA) [4–7] achieve excellent design automation for computer vision tasks, but require enormous computational resources hindering their widespread adoption. The computational expense of NAS is due to; (1) excessively large network design spaces, often containing billions of candidate architectures, (2) each candidate requiring training (which itself is expensive) to determine how it performs as compared to others, and 3) discrete search strategies attempting to evaluate a large number of candidates. Therefore, NAS research is highly focused on efficiency by adopting modular search instead of global [9, 15], and by approximating networks' performance instead of expensive training [8, 10, 15, 22, 23]. For details of NAS research, we refer the reader to [27].

Modular NAS, first introduced by NASNet [9] leads to efficient search, e.g., from several thousands [4] to 4 GPU-days [15, 23]. Hence, it has been widely researched by the NAS community, but has certain drawbacks. Modular NAS proposes to search only for a network module, also called a cell, rather than searching the entire architecture (global NAS). However, once a module is discovered for a given dataset, the total number of modules and channels (network's

Code and results available at: https://github.com/siddikui/Efficient-Macro-Micro-NAS.

✉ Shahid Siddiqui
msiddi01@ucy.ac.cy

Christos Kyrkou
kyrkou.christos@ucy.ac.cy

Theocharis Theocharides
ttheocharides@ucy.ac.cy

[1] Electrical and Computer Engineering Department, University of Cyprus, Nicosia, Republic of Cyprus

[2] KIOS Research and Innovation Center of Excellence, University of Cyprus, Nicosia, Republic of Cyprus

Published online: 13 March 2025

depth and width, i.e. macro architecture) is still unknown and needs manual intervention. This contradicts the main objective of automatically designing a suitable network for a given dataset. Another fundamental issue of modular search spaces is their narrow accuracy range and even randomly sampled architectures perform well [18]. Such spaces may yield good results quickly, but do not possess architectural diversity suitable for a variety of datasets. Therefore, in [47] we propose a flexible search space in terms of accuracy and network complexity. In this work, we further show how the design principles introduced in [47], can easily be adapted for real-world applications. More specifically, we expand on the concepts to develop ResNet [3] based search space for face recognition applications that offers more accurate and efficient networks than those used by leading face recognition methods [57, 58]. The same design principles can be used for other computer vision applications.

To establish relative rankings of networks, early methods [4] train and evaluate a large number of candidates (12,800 networks each for 50 epoch), which is extremely computationally intensive. Many subsequent works propose performance approximations instead of training each network from scratch. [8, 10] suggest reusing the weights of already trained networks during the search, but it becomes unclear whether the accuracy gain is due to the pre-trained weights or the superiority of the discovered architecture. Similarly, techniques based on parameter sharing in the supernet [15, 23] lead to inaccurate relative rankings as discussed in [26]. Many performance predictors have also been proposed [22], but training and evaluating predictors itself requires ground-truth validation accuracies of a large number of trained candidates, which is impractical to acquire for every new use case. To address this problem, we propose using a true ranking of the architectures in [47]. However, another issue is that all networks are trained under one training protocol, while training hyperparameter settings has been shown to have a greater effect on accuracy than the architecture itself [18, 28]. Keeping that in mind, we propose a novel ranking mechanism that assigns the training protocol relative to the learning capacity of the candidates, leading to a high correlation with their final performance compared to when they are all trained under similar settings. This allows a fairer and faster comparison of candidates, hence guiding the search strategy accurately and efficiently.

In terms of search strategy, NAS works treat the entire search space as one huge combinatorial space, therefore naturally apply discrete optimization algorithms [4–6, 9]. This usually necessitates evaluation of a large number of networks, hence a surge in the search cost. In this work, we split the search into macro and micro architecture discovery. In addition to reducing search costs, this allows the algorithm to first discover a good trade-off between network complexity and the given dataset difficulty. Once a good outer skeleton is learned, micro search can further enhance the fine grain-architecture. In summary, our main contributions are the following:

- A minimal but architecturally diverse search space.
- A more accurate and faster network ranking mechanism.
- An efficient search algorithm for end-to-end network discovery.
- Transferability of our framework's design principles to real-world computer vision applications.

Our proposed framework discovers competitive architectures in terms of both accuracy and size for CIFAR-10, CIFAR-100 [41] and FasionMNIST [42] while being 2× to 4× faster than the fastest global NAS method [8]. Moreover, compared to the best manually designed networks, we achieve new state-of-the-art results on EMNIST [43] and KMNIST [44]. For real-world applications of 1:1 face verification and top-n identification on a challenging Tinyface [56] dataset, we perform better than a leading face recognition method Adaface [58] with networks up to 2× smaller than commonly used ResNets [3], therefore bridging the gap between theory and applications.

Related Work

In this work, we reemphasize the automation of the network design process, therefore our work is closely related to the first work [13] that revisits NAS for end-to-end discovery with minimal human intervention. This line of work is further emphasized by the more recent work of [24], which proposes automatically generated search spaces from existing architectures. Another recent work is that of [23] as it focuses on both micro and macro search; however, it still manually stacks up the blocks to decide the final architecture. Further, we divide the related work into two parts; (1) neural architecture search and (2) face recognition applications.

Neural Architecture Search

Search Space

End-to-end NAS methods search for network depth, width, and convolutional kernel size [4–6, 8, 10, 11] except [7]. In addition, all these works search for skip connections except [5, 10]. Some works follow the VGG-like CONV-POOL-FC design paradigm [5, 10, 11]. We analyze these existing works in depth in Sect. Search Space Design and propose search space optimizations.

Search Strategy

The algorithms most commonly used by the research community are based on reinforcement learning (RL) [4, 5, 10], evolutionary algorithms [6–8, 24], and sequential model-based optimization (SMBO) [11, 19] methods. Due to the huge search spaces, it is computationally expensive for these methods to accurately evaluate a large number of candidates. In Sect. Search Algorithm, we propose an algorithm that can discover architectures efficiently.

Performance Evaluation

Early NAS works [4–7] fully train candidates to establish their relative ranking and employ speed-up strategies, but an excessively large number of architectures to be evaluated leads to enormous compute costs. Hence, a significant body of NAS research lies in the speed of candidate evaluation. Model-based accuracy predictors are fairly simple [31] but require architecture-accuracy paired data for every new dataset, which itself is expensive to obtain, and hence are not generalizable. [29] and [30] use learning curve prediction, and estimate the final rankings by training for fewer epochs, respectively. Although these methods may perform better for modular search spaces, our experiments in a global space show low correlation between early and final performance of the networks. The most widely adopted technique is perhaps parameter sharing in a supernet [14], where subgraphs can inherit weights from a large over-parameterized network. [26] shows that weight sharing leads to inaccurate rankings. Recently, there has been a substantial body of research using zero-cost proxies [32], which require no training or very little training. However, works such as [33, 34] show that proxies developed for modular search spaces are not transferable to global spaces and existing zero-shot proxies cannot outperform simple baseline such as number of parameters and FLOPS. In general, excessive focus on modular search has lead to a substantial research gap for efficient evaluation methods for global NAS.

NAS for Face Recognition

To demonstrate the transferability of our framework to face recognition applications, we use ResNets [3], as modified in [57, 58], as our baseline architectures. [59] applies joint NAS and hyperparameter optimization (HPO) to mitigate bias in face recognition. [60, 61] combine knowledge distillation and NAS to achieve low-complexity, high-accuracy networks. However, our method discovers high-accuracy low-complexity networks solely based on the NAS framework. Although relevant, these methods are highly specialized for face recognition and hence are not directly comparable to our NAS only method.

Methodology

In this section, we discuss our proposed search space, search strategy, and performance evaluation method.

Search Space Design

A **search space**, or just **space** from here on, is defined as a set of network variables from which various architectural configurations can be sampled. Table 1 shows that most existing spaces [4, 5, 10, 11] are influenced by the early Conv-Pool-FC-like architecture paradigm [1] and/or residual networks [3]. Moreover, network depth, width, and kernel size are the most common variables followed by convolution stride (Strides), skip connections, pooling layers, and fully connected (FC) layers. Since the number of network configurations grows exponentially with the number of search variables, we set the variables so that the resulting space is navigable for our search strategy. However, it is not sufficient

Table 1 Search Space Comparison: The proposed search space focuses on the most impactful design choices in terms of efficiency

NAS Method	Global Search Space Architectural Variables							
	Depth (Layers)	Width (Channels)	Operations per Layer	Convolutional Kernel	Strides	Pooling Layers	Fully Connected Layers	Skip Connections
NAS-RL [4]	✓	✓		✓	✓	✓		✓
Meta-QNN [5]	✓	✓		✓	✓	✓	✓	
Large-scale Evolution [6]	✓	✓		✓	✓			✓
EAS [10]	✓	✓		✓	✓	✓	✓	
Genetic Programming CNN [7]		✓		✓				✓
NASH-Net [8]	✓	✓		✓				✓
NASBOT [11]	✓	✓		✓	✓	✓	✓	✓
TRG-NAS [47]	✓	✓	✓	✓				

to just drop most of the variables for efficiency gains since this may limit the space in terms of performance. Hence, we strike a balance between automation and search space effectiveness, i.e. wide range performance/complexity trade-off. Such a space can better adapt to varying complexity tasks, by offering smaller networks for easier tasks and relatively complex networks for harder ones. Next, we discuss the optimizations done to create such a search space.

Trimming Search Variables

To start with, we can drop variables arising from early Conv-Pool-FC-like architectures [1] by leveraging FCN-like networks [2]. Therefore, FC layers can be replaced by a global pooling layer, and pooling layers can be replaced by convolutions with stride 2. Additionally, we can drop skip connections since we are not explicitly seeking very deep networks. Hence, we trim down **Fully connected**, **Pooling layers**, and **Skip connections** from Table 1.

Channels Search Reduction

Table 1 shows that all methods search for width (number of channels). This is usually done for each layer as in [4]. However, we limit the search for the channel to the initial layer only and use a fixed rate of channel doubling whenever the spatial dimensions are halved, as in [1, 15]. Additionally, it allows us to fix stride values of convolutions to 2 whenever doubling the channels and 1 otherwise. Hence, we drop **strides** from the search space. This technique further reduces the search complexity (discussed in the next section), but still allows variable-width architectural diversity.

Performance/Complexity Trade-off

We notice that existing global spaces do not allow operation search whereas operation type such as separable, dilated or plain convolution can allow significant expressiveness. Although we target a compact space, it should still maintain the idea of previously unseen and novel architectures. Hence, we allow searching for **Operation** type as either separable or plain convolution. For every layer, operation choice coupled with kernel choice of 3 or 5 creates a good number of possible micro architecture variations.

To this end, we propose a search space with ***depth***, ***width***, ***operations***, and ***kernels*** variables, as shown in Table 1. To study the accuracy variance of this space, we randomly sample 240 networks and train each network for 50 epochs on CIFAR-10. The worst network achieves an accuracy of 76.11%, while the best network is 94.65%. For comparison, the most widely adopted DARTS space has its worst network achieving 96.18% and the best 97.56% from within 214 sampled architectures by [18]. This shows that our search space has a high variance in terms of performance, therefore, better discovered networks can be attributed to the superiority of the search algorithm and not to expertly crafted space.

Search Space Complexity

The complexity of the space depends on search bounds and increases exponentially with depth. For a depth range of D, a width range of W, a number of operations O, a number of kernels K, and a final discovered depth D_f, the maximum number of architectures N_{arch} is given in the Eq. 1.

$$N_{arch} = (O \times K)^{D_f} \times D \times W \tag{1}$$

We limit the depth range between 5 to 100 layers, channels from 16 to 128 with steps of 2, two operations and two kernel sizes i.e., $D = 96$ and $W = 57, O = 2$ and $K = 2$. Then, assuming $D_f = 25$, the space as described above has approximately 6.16×10^{18} candidate architectures. For comparison, the search space for darts [15] is also approximately 10^{18} but has a much narrower accuracy range than ours. With the search variables explained, we can now formally define the search problem.

Search Problem

Let $\mathcal{L}_{train}$ and $\mathcal{L}_{test}$ denote training and test loss, respectively. These are determined by the network architecture x and its weights θ. The search goal is to find x^* that minimizes the test loss $\mathcal{L}_{test}(\theta^*, x^*)$, where the weights θ^* associated with the architecture are obtained by minimizing the training loss $\theta^* = \arg\min_{\theta} \mathcal{L}_{train}(\theta, x^*)$. This is a bilevel optimization problem with x as outer-level optimization variables and θ as inner-level optimization variables:

$$\min_{x \in \mathcal{X}} \mathcal{L}_{test}(\theta^*(x), x) \tag{2}$$

$$\textbf{s.t. } \theta^*(x) = \arg\min_{\theta} \mathcal{L}_{train}(\theta, x) \tag{3}$$

where $\mathcal{X} = \{(D, W, O, K) \mid D \in [D_{\min}, D_{\max}], W \in \{W_{\min} + ne \mid n \in \mathbb{N}_0, e \in E\}, O \in \{o_1, o_2\}, K \in \{k_1, k_2\}\}$. D, W, O and K determine network depth, width, operation type, and kernel size, respectively.

Performance Evaluation

Solving Eq. 3 by training each candidate from scratch and until convergence is expensive, hence previous work has used various approximation methods [8, 10, 15, 22]. In the development of these methods, it is assumed that all networks are comparable under a single training protocol, while

[18, 28] suggest that the training protocol may affect the accuracy more than the architecture itself [18, 28]. In this section, we show that ranking networks by training them differently (dynamic learning rankings), leads to more accurate relative rankings as compared to when they are all trained under similar settings (static learning rankings).

Static Learning Rankings

We randomly select 240 networks from our search space, train each for 50 epochs on CIFAR-10 following [4], and record their validation accuracies at every epoch. We calculate Spearman rank correlation metric [35] between 'models trained for 1 epoch' and 'the same models trained for 50 epochs' i.e. their final performance. We find that the correlation value is *0.65*, indicating that ***rankings of networks determined by shorter training are not as accurate as when fully trained***. Then, we calculate the same metric between 'number of parameters of models' or simply **params** and their final performance. The correlation value turns out to be *0.49*, showing that ***the number of parameters alone are not indicative of the final performance of the network***, i.e. larger networks are not necessarily better networks.

Dynamic Learning Rankings

Next we sample 50 networks from 240 and sort w.r.t. their parameters in ascending order. For each network increasing in parameters, we record its validation accuracy against training for an additional epoch. For example, the smallest network trained only for 1 epoch, the next one for 2 epochs, and so on. We observe a high correlation value of *0.91* between params and corresponding 'validation accuracy with increasing epochs' (dynamic learning rankings). This indicates that ***a candidate may perform better relative to others due to increased parameters or better training, or both***. In addition, networks ranked using dynamic learning have a high correlation value of *0.85* with their final performance, ie *20%* better than static learning rankings. Therefore, ***not all networks need to be trained until convergence and their training protocol can be chosen relative to each other***.

Applicability to NAS

We use the insights gained from the above experiments and employ a dynamic learning evaluation mechanism to determine relative rankings of candidates during the search as follows: (1) Whenever we evaluate a candidate with more parameters compared to its neighbor, we also train it for an additional epoch, (2) However, while comparing model with lesser parameters compared to its neighbor, we expect to improve performance due to better training, so we train models even longer, (3) Finally, we follow consistent training when comparing models with similar number of parameters. In this way, we are able to achieve better relative ranking without fully training all models, hence a much faster evaluation scheme.

Search Algorithm

The proposed search space is still large; however, we have set the architectural variables in such a way that we can split the global search into macro and micro searches, enabling our algorithm 1 to navigate it efficiently.

Macro Architecture Search

We initialize the search with a small number of evaluation epochs E_{min}, minimum depth D_{min}, maximum width W_{max} and all operations and kernel sizes set to separable convolution (Sep) and 3×3, respectively. We let candidates ***Grow*** layers in an attempt to achieve better validation accuracy. As discussed in section Performance Evaluation, for each added layer, the candidates are trained for an additional epoch compared to the previous one. Layers are added until they increase performance by $L^{+}_{acc^{+}}$(accuracy gain by adding layer). By adding a layer, if the accuracy does not drop more than $L^{+}_{acc^{-}}$(accuracy drop by adding layer), we continue searching for depth. The depth search is terminated if D_{max} (upper bound of layers) is reached or the accuracy drops more than $L^{+}_{acc^{-}}$ three times. We empirically determine that the threshold values for $L^{+}_{acc^{+}}$ and $L^{+}_{acc^{-}}$ are 0.10 and 0.05, respectively. Once the depth search ends, we ***Prune*** the number of channels until W_{min} is reached or the accuracy drops below the previous best three times. Since we are decreasing the learnable parameters when pruning, we expect the performance to drop, therefore, for every pruned candidate, we evaluate it with 2 additional training epochs, as mentioned in Sect. Performance Evaluation. After the width search ends, we have an architecture with D_f and W_f, i.e., final depth and width, respectively, while O and K remain unchanged. The macro search discovers a good depth and width for the given dataset while reducing the right term of complexity in the Eq. 1 to $D'+W'$, where D' and W' are the number of networks evaluated for depth and width, respectively.

Algorithm 1 NAS Search Algorithm

Input: Search bounds: D_{min}, D_{max}, W_{min}, W_{max}, W_{res}
Initialization: $E = E_{min}$, $L = D_{min}$, $C = W_{max}$, $O = Sep$, $K = 3 \times 3$
MACRO Search
1. ***Grow*** model $L \leftarrow L + 1$ & **train** for $E \leftarrow E + 1$ ***while*** Acc^{test} improves
2. ***Prune*** model $C \leftarrow C - W_{res}$ & **train** for $E \leftarrow E + 2$ ***while*** Acc^{test} retains
return ***Macro*** *architecture* D_f, W_m, O, K
MICRO Search
$E = E_{min}$, $L = D_f$, $C = W_m$, $O = Sep$, $K = 3 \times 3$
1. *Retrain* ***Macro*** architecture for E_{min} as new baseline
2. ***Replace*** $O_i \leftarrow Conv$ & C-- ***if*** Acc^{test} improves by **training** for E_{min}
3. ***Update*** $K_i \leftarrow [5 \times 5]$ & C-- ***if*** Acc^{test} improves by **training** for E_{min}
Return *global architecture with* D_f, W_f, O_f, K_f

Micro Architecture Search

Macro search adapts the architecture to a good performance point. We subsequently try to fine-tune it with operation type and kernel size at each layer. However, in addition to the micro space being combinatorially huge, see Eq. 1, plain convolution and larger kernel size drastically increase network parameters. This is problematic in the sense that we would like to achieve better performance as a result of better architectures and not due to increased parameters. Therefore, for micro search, whenever we use plain convolution or larger kernel size, we decrease the number of channels such that the total number of parameters remains approximately equal to that of baseline macro architecture. There are two benefits of this approach, 1) we can compare all micro architectures with an equal and small number of training epochs as discussed in Sect. Performance Evaluation, 2) we can evaluate a reasonably high number of micro architectures. Therefore, we initiate micro search by retraining the discovered macro architecture with E_{min} as a baseline and then perform search operations and kernel sizes for each layer. We ***Replace*** separable convolutions with plain ones and ***Update*** kernel sizes if the validation accuracy improves by training for E_{min}. Micro search ends once all layers are traversed for alternative operations and kernels. Hence, we discover O_f and K_f by evaluating $2 \times D_f$ architectures. At this point, we have adapted an architecture for the target dataset by evaluating only $N_{evaluated} = 2 \times D_f + D' + W'$ architectures instead of the number shown in Eq. 1.

Parameter Efficient Networks

As shown in Algorithm 1, we initialize the search with minimum depth, maximum width, and all layers of separable convolutions with kernel sizes of 3×3. This decision is reached by empirically evaluating alternative initialization strategies where layers can initially be convolutions or kernel sizes be 7×7, as shown in Table 2. To single out the contribution of each strategy and for faster evaluation, we sample 10 binary subdatasets from CIFAR-10 instead of using the entire dataset and record the averaged accuracy and number of parameters. In Table 2, we show that the best strategy is to start with smaller networks and add parameters only if performance improves. This strategy significantly beats others in terms of the accuracy/parameter efficiency trade-off.

Experiments and Results

This section is divided into two subsections. Section Image Classification Experiments and Results is focused on experiments and results for image classification datasets. Section Face Recognition Experiments and Results details

Table 2 Effect of different initialization strategies on search

Initialization Strategy	Conv-64-3x3	Conv-64-7x7	Sep-64-3x3	Sep-64-7x7
Accuracy (%)	97.85	97.35	**97.96**	97.73
Parameters (M)	0.65	0.64	**0.23**	0.90

Bold represents the best results

the transferability of our framework to facial recognition applications.

Image Classification Experiments and Results

Datasets

For comparison with NAS methods, we use the CIFAR-10 [41], CIFAR-100 [41], and FashionMNIST [42] datasets. To compare with the previous manually designed state-of-the-art approaches, we use EMNIST balanced [43] and KMNIST [44] datasets.

Search and Training Settings

For all datasets, we perform the search under two settings i.e. **tiny** and **mobile** networks. For **tiny** settings, we run search with D_{min}=10, D_{max}=100, W_{min}=16, W_{max}=64, W_{res}=2 and E_{min} =10 for macro search and E_{min}=2 for micro search. For **mobile** settings, we simply increase the W_{max}=128. During the search, we train all candidates with varying epochs (as discussed in our search strategy) using SGD with momentum of 0.9 and weight decay of 3e-4. We used an initial learning rate of 0.025 annealed down to 0 using a cosine scheduler, batch size of 64 and cutout augmentation [40]. For final training, we use the same settings as search except epochs, which are fixed at 600. In general, we use standard training settings as in [15, 24] and do not use enhanced training protocols that hide the contributions of the search strategy or the search space [18]. To show the contributions of our proposed method, we follow the best practices of NAS as suggested by [18, 20]. All experiments were carried out on a single Nvidia Quadro RTX 8000 GPU.

Table 4 Comparison with state-of-the-art architectures for CIFAR-100

Method	Test Err. (%)	Params. (M)	Search Cost (GPU-days)	Search Space	Search Strategy
ResNet [3]	29.14	1.7	manual	–	–
Macro-NAS [24]	25.45	51.5	2.7	Global	EA
Ours (tiny)	**23.64**	**0.49**	**0.17**	Global	Greedy
Ours (mobile)	**22.04**	**2.49**	**0.33**	Global	Greedy

Bold represents the best results

Table 3 Comparison with state-of-the-art NAS architectures for CIFAR-10

NAS Method	Test Err. (%)	Params (M)	Search Cost (GPU-days)	Search Space	Search Algorithm
ResNet [3]	6.43	1.7	-	-	-
NAS-RL [4]	3.65	37.4	22400	Global	RL
Meta-QNN [5]	6.92	11.2	100	Global	RL
EAS [10]	4.23	23.4	10	Global	RL
Large-scale Evolution [6]	5.40	5.4	2600	Global	EA
Genetic Programming CNN [7]	5.98	1.7	14.9	Global	EA
NASH-Net [8]	5.20	19.7	1	Global	EA
Macro-NAS [24]	4.23	6.7	1.03	Global	EA
RandGrow [13]	3.38	3.1	6	Global	RS
Petridish [17]	2.83	2.2	5	Global	Gradient
NASBOT [11]	8.69	N/A	1.7	Global	SMBO
NSGA-NET [19]	3.85	3.3	8	Global	SMBO
NASNet-A [9]	2.65	3.3	2000	Modular	RL
pEvoNAS-C10A [25]	2.48	3.6	1.20	Modular	EA
DPP-Net [12]	5.84	0.45	2	Modular	SMBO
DARTS [15]	2.76	3.3	4	Modular	Gradient
GDAS [16]	2.82	2.5	0.17	Modular	Gradient
AGNAS [23]	2.46	3.6	0.4	Modular	Gradient
NAS-Bench-201 (Best) [37]	5.63	1.1	–	Modular	-
SGAS [46]	2.66 ± 0.24	3.7	0.25	Modular	Gradient-Greedy
TRG-NAS [47]	4.00	0.45	4.5	Global	Greedy
Random (Ours)	6.95 ± 2.18	0.77 ± 0.70	–	Global	–
Ours (tiny)	**4.09**	**0.46**	**0.24**	Global	Greedy
Ours (mobile)	**3.17**	**2.49**	**0.43**	Global	Greedy

Bold represents the best results

CIFAR Results

We now discuss our results on CIFAR-10 and CIFAR-100. CIFAR-10 is the most widely used dataset for NAS. So, although our work is related to global NAS, for the sake of completeness, we also compare against modular strategies, as shown in Table 3. Our approach with **tiny** settings achieves a 4.09% error rate with a small 0.46M parameter model in just 0.24 GPU days. Further, our model size is roughly equal to that of DPP-Net [12] and TRG-NAS (0.45M), but we achieve 1.84% higher accuracy than the former and are 18× faster than later, respectively. Our network in **mobile** settings is only outperformed by Petridish among global methods, but our search is 11× faster. For CIFAR-100, we compare with Macro-NAS [13], the most closely related and recent global NAS method, and also with a widely used manually designed network ResNet [3]. Both of our **tiny** and **mobile** models outperform the best global NAS method and the manually designed network in terms of precision, parameter efficiency and search cost, as shown in Table 4. Moreover, our **tiny** model is 105× smaller than Macro-NAS [13], while being 1. 81% more accurate and at 15× faster search. In general, our approach offers a better trade-off between automatic network design, high accuracy, lower model complexity, and faster search from 2× to 4×.

MNIST Results

We achieve state-of-the-art results on EMNIST (balanced) and KMNIST datasets as compared to best manually designed networks as shown in Table 5. For **EMNIST**, our tiny model is 6× smaller than SOTA WaveMix [36] (which is a highly parameter-efficient network design method), and 0.07% better, while our mobile model

Table 5 Comparison with state-of-the-art for FashionMNIST, EMNIST, and KMNIST

Method	EMNIST			KMNIST			FashionMNIST		
	Acc. (%)	Params (M)	Cost (Days)	Acc. (%)	Params (M)	Cost (Days)	Acc. (%)	Params (M)	Cost (Days)
WaveMix [36]	91.06	2.42	manual	–	–	–	–	–	–
u2Net [39]	–	–	–	98.68	300	manual	–	–	–
FT-DARTS [38]	–	–	–	–	–	–	**96.91**	3.2	1
Ours (tiny)	**91.20**	**0.40**	**0.36**	**99.35**	**0.42**	0.36	95.90	**0.50**	**0.11**
Ours (mobile)	**91.48**	**2.25**	**0.94**	**99.29**	**2.71**	0.60	96.22	6.15	**0.13**

Bold represents the best results

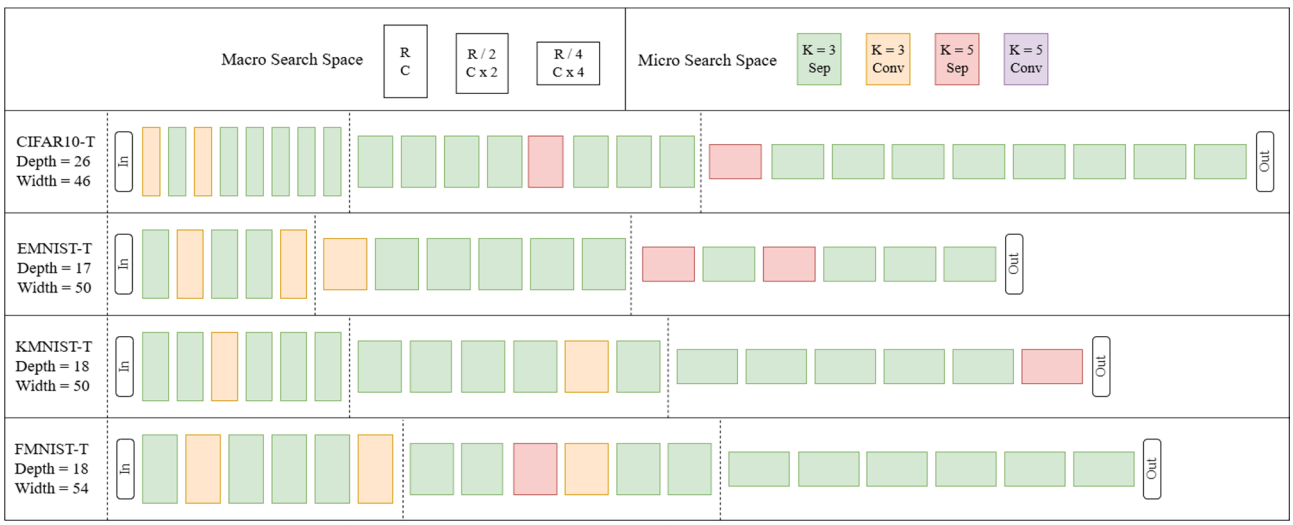


Fig. 1 Tiny (T) models discovered for CIFAR-10, EMNIST, KMNIST and FashionMNIST datasets. Block height and width represents image resolution (R) and number of channels (C) respectively, while depth is represented by the total number of blocks. Different colors in Micro Search Space represent possible operation types and kernel sizes. Due to space limitations, we do not show CIFAR-100 and mobile networks

achieves 0.20% higher accuracy. For **KMNIST**, our tiny model performs 0.67% better than the previous SOTA ViT-L/16 transformer-based network u2Net [39] having 300 M parameters. For **FashionMNIST**, we compare our results with the state-of-the-art NAS method FT-DARTS [38], although it is a modular search. Our search is 9× faster and our **tiny** model is 6× smaller than FT-DARTS. Furthermore, it utilizes specialized training protocols such as drop path and random erasing. Besides this, our results with mobile network are still very competitive in terms of accuracy too without making use of advanced techniques, with the performance stemming from our proposed approach. Figure 1 shows a few networks discovered by our framework.

Ablation Studies

As pointed by [18], even randomly sampled architectures perform well in modular search spaces. Therefore, it cannot be determined whether the better discovered networks are due to expertly crafted search spaces or due to superior search strategies. To effectively single out the contribution of each NAS component, [18] suggests comparing against a simple baseline of ***Randomly Sampled*** architectures and use ***Relative Improvement*** metric (RI). Therefore, to show the effectiveness of our search strategy, we compare it with 10 randomly sampled architectures for CIFAR-10. In Table 3, we show that our approach achieves 2.86% less error with 31% fewer parameters on average. This clearly singles out the contribution of our algorithm. In addition, we use the RI metric, which is $RI = 100 \times (Acc_m - Acc_r)/Acc_r$, where Acc_m and Acc_r represent the accuracy of the search method and the average accuracy of the architecture sampled randomly, respectively. According to [18], a good search strategy should achieve $RI > 0$ across different runs. Our method consistently achieves an $RI > 2$ in 5 different search runs for CIFAR-10. We also test our search space with and without operations and run search for both scenarios. We used 10 different seeds for each scenario and trained candidates for 20 epochs for faster search. Searching with operations, on average, yields 0.79% higher mean accuracy than searching without operations, that is, 89. 92% and 89. 13%, respectively. When searched for 600 epochs, the resulting best model without operations achieves an accuracy of 95.82% as compared to 96% with operations, therefore showing the contribution of operations in the search space.

Face Recognition Experiments and Results

Datasets

For search and training, we use the CASIA-WebFace dataset [48]. For evaluation, we use three popular categories of test datasets; **1. High Quality** LFW [49], CFP-FP [50], CPLFW [51], AgeDB [52] and CALFW [53], **2. Mixed Quality** IJB-B and IJB-C [54, 55], and **3. Low Quality** [56].

Search and Training Settings

We use the same ranking mechanism and search algorithm presented in Sects. Performance Evaluation and Search Algorithm, respectively. We use the exact search settings as mentioned in Sect. Search and Training Settings, with the exception of W_{max}=32 for **small** and W_{max}=64 for **medium** settings. However, for a fair comparison with Adaface [58], we slightly modify our search space to use an architecture based on inverted ResNet [3]. Note that the fundamental search variables are the same, that is, depth, width, operations, and kernels as introduced in Sect. Search Space Design. This further validates the transferability of our framework to applications beyond theory. However, due to a large number of training samples i.e. 0.5M and higher spatial resolution of 112x112, the search takes around 4 and 6 GPU days, for our small and medium models respectively, hence we early stop at best networks discovered by macro search only. For training, we follow the same protocol as in [58].

High Quality Results

For small settings, we discover a model with 58 layers but half the number of channels used by [58], that is, 256 in final block compared to 512. Our model performs better on all high-quality datasets for the 1:1 verification task, as shown in Table 6. A comparison of our small model and

Table 6 1:1 verification accuracy and model parameters (in million; M) comparison of Adaface and our searched models on high quality datasets

Method	Model	Params. (M)	High quality test data					
			LFW [49]	CFP-FP [50]	CPLFW [51]	AgeDB [52]	CALFW [53]	AVG
Adaface [58]	R-18-512	29.4	99.10	92.91	85.97	92.93	92.67	92.72
Ours(small)	R-58-256	**14.6**	**99.33**	**93.80**	**88.60**	**93.43**	**92.93**	**93.62**
Adaface [58]	R-50-512	49.0	**99.40**	94.71	89.33	93.88	93.33	94.13
Ours(medium)	R-34-512	**37.1**	99.35	**95.13**	**89.83**	**94.27**	**93.68**	**94.45**

Bold represents the best results

Table 7 For mixed quality datasets IJB-B and IJB-C, TAR@FAR=0.01% are reported. For low quality dataset TinyFace, Top-1, Top-5 and Top-20 rank retrieval is used

Method	Model	Params (M)	Mixed Quality		Low Quality		
			IJB-B [54]	IJB-C [55]	TinyFace [56]		
			TAR@FAR=0.01%	TAR@FAR=0.01%	Rank-1	Rank-5	Rank-20
Adaface [58]	R-18-512	29.4	**93.11**	**94.03**	54.85	60.30	64.10
Ours(small)	R-58-256	**14.6**	92.06	92.60	**57.78**	**62.66**	**66.36**
Adaface [58]	R-50-512	49.0	**94.52**	**95.34**	57.32	62.63	66.66
Ours(medium)	R-34-512	**37.1**	93.22	93.65	**58.66**	**64.54**	**67.97**

Bold represents the best results

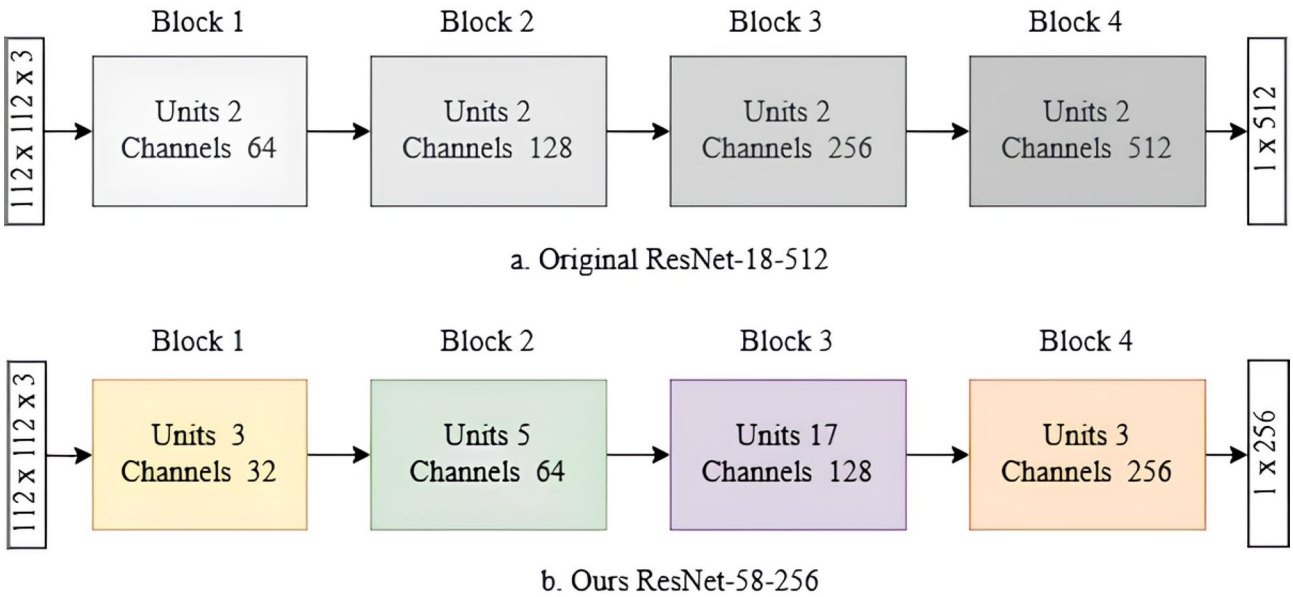


Fig. 2 Architectural comparison of a. R-18-512 with our b. R-58-256

ResNet-18 is shown in Fig. 2. Ours has 2× fewer parameters than ResNet-18. For medium settings, we discover a model with just 34 layers, having 1.3× fewer parameters as compared to ResNet-50, but still performing better on all datasets except LFW.

Mixed and Low Quality Results

Table 7 shows our results on mixed and low-quality test datasets. Given that our models are much smaller, we achieve competitive results on both IJB-B and IJB-C datasets [54, 55]. On Tinyface [56], both of our models achieve better accuracy in the rank-1, rank-5, and rank-20 identification task.

Limitations and Future Work

Although, we have used face recognition to demonstrate real-world applications of our framework, in general, given enough computational resources, it can yield accurate and parameter-efficient models for other computer vision applications too. Moreover, since we are searching on the whole dataset to evaluate candidates, the dataset size and image resolution influence candidates' training time, and hence search speeds. Therefore, further speedups can be investigated in the context of larger datasets such as ImageNet [45] and CASIA-Webface [48] leveraging data subsets instead of entire data. Moreover, we observe that macro search coupled with good training protocols achieves more significant accuracy gains compared to micro search. Therefore, another promising research direction is to redesign the framework for joint macro architecture and training protocol search to improve overall accuracy and search efficiency.

Conclusion

Contrary to trending modular search that offers partial network discovery, we revisit NAS for end-to-end network discovery. Our search space offers networks of varying complexity and is designed such that it can be efficiently navigated by the proposed search strategy. Our architecture aware ranking mechanism leads to much faster search as compared to existing methods, yet consistently assists in discovering highly accurate tiny and mobile architectures. Lastly, we demonstrate the transferability of our framework from theory to real-world applications of face verification and identification with the continued trend of discovering more accurate, yet smaller networks.

Author contributions Shahid Siddiqui designed the NAS framework, conducted the experiments, and wrote the initial draft of this manuscript. Christos Kyrkou suggested experiments to test the framework and further improved the draft. Theocharis Theocharides provided guidance for the final manuscript.

Funding Open access funding provided by the Cyprus Libraries Consortium (CLC). This work was supported by the European Union' Horizon 2020 research and innovation program under grant agreement No 739551 (KIOS CoE) and from the Government of the Republic of Cyprus through the Directorate General for European Programs, Coordination and Development.

Data availability All search and training codes, as well as logs are available at: https://github.com/siddikui/Efficient-Macro-Micro-NAS.

Declarations

Conflict of interest The authors declare that they have no Conflict of interest.

Research Involving Humans and/or Animals Not applicable.

Informed Consent Informed consent was obtained from all participants included in the study.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Simonyan Karen, Zisserman Andrew. Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556 (2014).
2. Long Jonathan, Shelhamer Evan, Darrell Trevor. Fully convolutional networks for semantic segmentation. Proceedings of the IEEE conference on computer vision and pattern recognition. 2015.
3. He Kaiming et al. Deep residual learning for image recognition. Proceedings of the IEEE conference on computer vision and pattern recognition. 2016.
4. Zoph Barret, Le Quoc V. Neural architecture search with reinforcement learning. arXiv preprint arXiv:1611.01578 (2016).
5. Baker Bowen et al. Designing neural network architectures using reinforcement learning. arXiv preprint arXiv:1611.02167 2016.
6. Real Esteban et al. Large-scale evolution of image classifiers. International Conference on Machine Learning. PMLR, 2017.
7. Suganuma Masanori, Shirakawa Shinichi, Nagao Tomoharu. A genetic programming approach to designing convolutional neural network architectures. Proceedings of the genetic and evolutionary computation conference. 2017.
8. Elsken Thomas, Metzen Jan-Hendrik, Hutter Frank. Simple and efficient architecture search for convolutional neural networks. arXiv preprint arXiv:1711.04528 2017.
9. Zoph Barret et al. Learning transferable architectures for scalable image recognition. Proceedings of the IEEE conference on computer vision and pattern recognition. 2018.
10. Cai Han et al. Efficient architecture search by network transformation. Proceedings of the AAAI Conference on Artificial Intelligence. 2018;32(1).
11. Kandasamy Kirthevasan et al. Neural architecture search with bayesian optimisation and optimal transport. Adv Neural Inf Proc Syst 2018;31.
12. Dong Jin-Dong et al. Dpp-net: Device-aware progressive search for pareto-optimal neural architectures. Proceedings of the European Conference on Computer Vision (ECCV). 2018.
13. Hu Hanzhang et al. Macro neural architecture search revisited. 2nd Workshop on Meta-Learning at NeurIPS. 2018.
14. Pham H, Guan M, Zoph B, Le Q, Dean J. Efficient neural architecture search via parameters sharing. InInternational conference on machine learning 2018;4095-4104.
15. Liu Hanxiao, Simonyan Karen, Yang Yiming. Darts: Differentiable architecture search. arXiv preprint arXiv:1806.09055 (2018).
16. Dong Xuanyi, Yang Yi. Searching for a robust neural architecture in four gpu hours. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2019.
17. Hu Hanzhang et al. "Efficient forward architecture search." Adv Neural Inf Proc Syst 32 (2019).
18. Yang Antoine, Pedro M. Esperança, and Fabio M. Carlucci. NAS evaluation is frustratingly hard. arXiv preprint arXiv:1912.12522 (2019).
19. Lu Zhichao et al. Nsga-net: neural architecture search using multi-objective genetic algorithm. Proceedings of the genetic and evolutionary computation conference. 2019.
20. Lindauer Marius, Hutter Frank. Best practices for scientific research on neural architecture search. J Machi Learn Res. 2020;21(1):9820–37.
21. Ren Pengzhen, et al. A comprehensive survey of neural architecture search: challenges and solutions. ACM Comput Surveys (CSUR). 2021;54(4):1–34.
22. White Colin, et al. How powerful are performance predictors in neural architecture search? Adv Neural Inf Proc Syst. 2021;34:28454–69.

23. Sun Zihao et al. “AGNAS: Attention-Guided Micro and Macro-Architecture Search.” International Conference on Machine Learning. PMLR, 2022.
24. Lopes Vasco, Luís A. Alexandre. "Towards Less Constrained Macro-Neural Architecture Search." arXiv preprint arXiv:2203.05508 (2022).
25. Sinha Nilotpal, Chen Kuan-Wen. “Neural architecture search using progressive evolution.” Proc Gen Evolut Comput Conf. 2022.
26. Yu Kaicheng, Sciuto Christian, Jaggi Martin, Musat Claudiu, Salzmann Mathieu. Evaluating the search phase of neural architecture search. In International Conference on Learning Representations, 2019.
27. White Colin, Safari Mahmoud, Sukthanker Rhea, Ru Binxin, Elsken Thomas, Zela Arber, Dey Debadeepta, Hutter Frank. “Neural architecture search: Insights from 1000 papers.” arXiv preprint arXiv:2301.08727 2023.
28. Bansal A, Stoll D, Janowski M, Zela A, Hutter F. Jahs-bench-201: a foundation for research on joint architecture and hyperparameter search. Adv Neural Inf Proc Syst. 2022;6(35):38788–802.
29. Baker B, Gupta O, Raskar R, Naik N. Accelerating neural architecture search using performance prediction. arXiv preprint arXiv:1705.10823. 2017.
30. Zheng X, Ji R, Tang L, Zhang B, Liu J, Tian Q. Multinomial distribution learning for effective neural architecture search. In Proceedings of the IEEE International Conference on Computer Vision 2019;1304-1313.
31. Wen Wei, Liu Hanxiao, Li Hai, Chen Yiran, Bender Gabriel, Kindermans Pieter-Jan. Neural predictor for neural architecture search. arXiv preprint arXiv:1912.00848, 2019.
32. Krishnakumar A, White C, Zela A, Tu R, Safari M, Hutter F. NAS-Bench-Suite-Zero: accelerating research on zero cost proxies. Adv Neural Inf Proc Syst. 2022;6(35):28037–51.
33. Chen Hanlin, Lin Ming, Sun Xiuyu, Li Hao. Nas-bench-zero: A large scale dataset for understanding zero-shot neural architecture search. Openreview preprint https://openreview.net/forum?id=hP-SILoczR, 2021.
34. Xuefei Ning, Changcheng Tang, Wenshuo Li, Zixuan Zhou, Shuang Liang, Huazhong Yang, Wang Yu. Evaluating efficient performance estimators of neural architectures. Adv Neural Inf Proc Syst. 2021;34:12265.
35. Spearman C. The proof and measurement of association between two things. Am J Psychol. 1904;15(1):72–101.
36. Jeevan P, Viswanathan K, Sethi A. Wavemix-lite: A resource-efficient neural network for image analysis. arXiv preprint arXiv:2205.14375. 2022.
37. Dong Xuanyi, Yang Yi. NAS-Bench-201: Extending the Scope of Reproducible Neural Architecture Search. In ICLR, 2020.
38. Tanveer MS, Karim Khan MU, Kyung C-M. “Fine-Tuning DARTS for Image Classification,” 2020 25th International Conference on Pattern Recognition (ICPR), Milan, Italy, 2021;4789-4796, https://doi.org/10.1109/ICPR48806.2021.9412221.
39. Gesmundo A, Dean J. An evolutionary approach to dynamic introduction of tasks in large-scale multitask learning systems. arXiv preprint arXiv:2205.12755. 2022.
40. DeVries Terrance, Taylor Graham. Improved Regularization of Convolutional Neural Networks with Cutout. 2017.
41. https://www.cs.toronto.edu/~kriz/cifar.html
42. Xiao H, Rasul K, Vollgraf R. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. arXiv preprint arXiv:1708.07747. 2017.
43. Cohen G, Afshar S, Tapson J, Van Schaik A. EMNIST: Extending MNIST to handwritten letters. In2017 international joint conference on neural networks (IJCNN) 2017;2921-2926. IEEE.
44. Clanuwat T, Bober-Irizar M, Kitamoto A, Lamb A, Yamamoto K, Ha D. Deep learning for classical japanese literature. arXiv preprint arXiv:1812.01718. 2018.
45. Russakovsky O, Deng J, Su H, Krause J, Satheesh S, Ma S, Huang Z, Karpathy A, Khosla A, Bernstein M, Berg AC. Imagenet large scale visual recognition challenge. Int J Comput vision. 2015;115:211–52.
46. Li G, Qian G, Delgadillo IC, Muller M, Thabet A, Ghanem B. Sgas: Sequential greedy architecture search. InProceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition 2020;1620-1630.
47. Siddiqui S, Kyrkou C, Theocharides T True Rank Guided Efficient Neural Architecture Search for End to End Low-Complexity Network Discovery. In: Tsapatsoulis, N., et al. Computer Analysis of Images and Patterns. CAIP 2023. 2023.
48. Yi D, Lei Z, Liao S, Li SZ. Learning face representation from scratch. arXiv preprint arXiv:1411.7923. (2014)
49. Huang GB, Mattar M, Berg T, Learned-Miller E. Labeled faces in the wild: A database forstudying face recognition in unconstrained environments. InWorkshop on faces in’Real-Life’Images: detection, alignment, and recognition 2008.
50. Sengupta S, Chen JC, Castillo C, Patel VM, Chellappa R, Jacobs DW. Frontal to profile face verification in the wild. In2016 IEEE winter conference on applications of computer vision (WACV) 2016;1-9. IEEE.
51. Zheng T, Deng W. Cross-pose lfw: a database for studying cross-pose face recognition in unconstrained environments. Beijing Univ Posts Telecommun Tech Rep. 2018;5(7):5.
52. Moschoglou S, Papaioannou A, Sagonas C, Deng J, Kotsia I, Zafeiriou S. Agedb: the first manually collected, in-the-wild age database. Inproc IEEE Vonf Comput Vision Pattern Recogn workshops. 2017;51:59.
53. Zheng T, Deng W, Hu J. Cross-age lfw: A database for studying cross-age face recognition in unconstrained environments. arXiv preprint arXiv:1708.08197. (2017).
54. Whitelam C, Taborsky E, Blanton A, Maze B, Adams J, Miller T, Kalka N, Jain AK, Duncan JA, Allen K, Cheney J. Iarpa janus benchmark-b face dataset. Inproceedings of the IEEE conference on computer vision and pattern recognition workshops 2017;90-98.
55. Maze B, Adams J, Duncan JA, Kalka N, Miller T, Otto C, Jain AK, Niggel WT, Anderson J, Cheney J, Grother P. Iarpa janus benchmark-c: Face dataset and protocol. In 2018 international conference on biometrics (ICB) 2018;158-165. IEEE.
56. Cheng Z, Zhu X, Gong S. Low-resolution face recognition. Cham: Springer International Publishing; 2019.
57. Deng J, Guo J, Xue N, Zafeiriou S. Arcface: Additive angular margin loss for deep face recognition. InProceedings of the IEEE/CVF conference on computer vision and pattern recognition 2019;4690-4699.
58. Kim M, Jain AK, Liu X. Adaface: Quality adaptive margin for face recognition. InProceedings of the IEEE/CVF conference on computer vision and pattern recognition 2022 ;18750-18759.
59. Dooley S, Sukthanker R, Dickerson J, White C, Hutter F, Goldblum M. Rethinking bias mitigation: Fairer architectures make for fairer face recognition. Adv Neural Inf Proc Syst. 2024;13:36.
60. Wang X. Teacher guided neural architecture search for face recognition. InProc AAAI Conf Artif Intell. 2021;35(4):2817–25.
61. Boutros F, Siebke P, Klemt M, Damer N, Kirchbuchner F, Kuijper A. Pocketnet: Extreme lightweight face recognition network using neural architecture search and multistep knowledge distillation. IEEE Access. 2022;26(10):46823–33.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.